EliteGuard Gateway

HTTP & gRPC Setup — Complete Implementation Guide

Sample services · Gateway in the middle · HTTP & gRPC forwarding · DB route storage

Scope covered by this guide

Requirement	What you get
Sample HTTP services	http-user (:9001), http-order (:9002)
Sample gRPC service	grpc-hello (:50052)
Gateway in the middle	Clients use :8080 (HTTP) and :50051 (gRPC)
HTTP forwarding	Dynamic reverse proxy per DB route
gRPC forwarding	Transparent gRPC proxy
DB storing routes	Postgres routes + upstreams, admin CRUD, gateway reload

Project	CoreGuard Gateway / EliteGuard (Go module: elitegate)
Version	1.0 — May 2026
Phases	16 implementation phases with full source code

Table of Contents

Recommended order: Phases 1–2 → 3 → 6 → 9 (HTTP demo) → 4–7 (gRPC) → 8 (Admin API)

Project: CoreGuard Gateway / EliteGuard (`module elitegate`) **Goal:** Sample HTTP + gRPC backends, API gateway in the middle, DB-backed routes, production-style Docker deployment **Version:** 1.0 — May 2026

Table of Contents

---

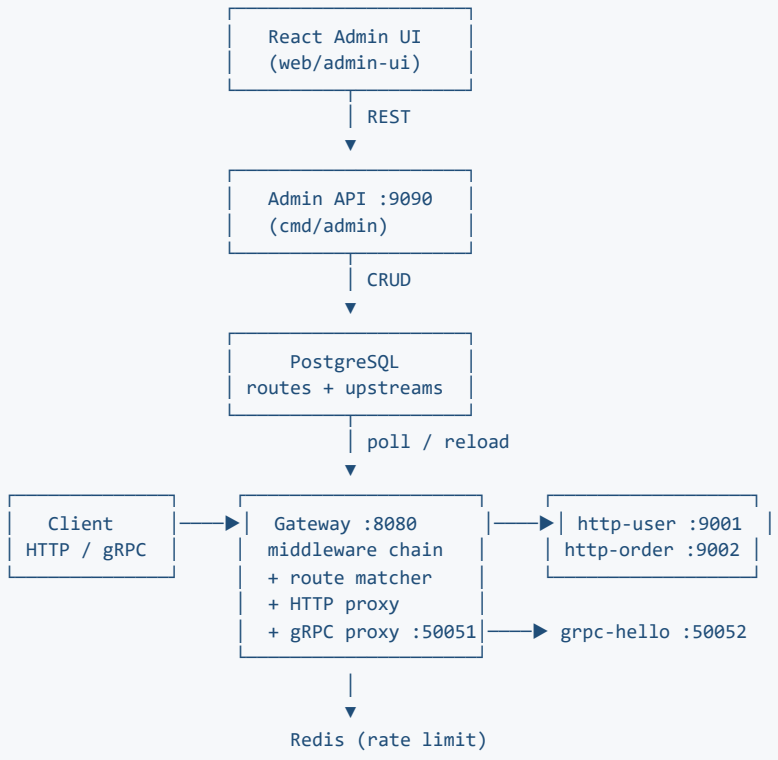
1. Current State vs Target

Area	Today	After this guide
HTTP forwarding	Single proxy → `AdminAPIURL`	Per-route proxy → sample/user/order services
gRPC forwarding	Empty `grpc_proxy.go`	Transparent HTTP/2 gRPC proxy on `:50051`
DB `routes` table	Schema only, admin stubs	Full CRUD + gateway loads from DB
DB `upstreams`	Missing migration	`upstreams` table + FK from routes
Sample services	`cmd/testbackend` on `:9090` (conflicts with admin)	Dedicated HTTP `:9001`, `:9002` + gRPC `:50051`
Admin UI	Calls `/routes`, `/upstreams` (stubs)	Same APIs, real persistence

Important: Gateway must **not** proxy public `/api/*` traffic to the admin service. Admin stays on `:9090` under `/admin/*` only.

---

2. Target Architecture



Request flow (HTTP):

```
Client → :8080/api/users → Logger → IP → JWT/API-Key → RateLimit
      → Match route "/api/users" → ReverseProxy → http-user-service:9001
```

Request flow (gRPC):

```
grpcurl → :50051 → gRPC proxy → grpc-hello-service:50052
```

---

3. Implementation Phases Overview

Phase	Effort	Delivers
1 DB schema	~1h	`upstreams`, extended `routes`
2 Repositories	~2h	Postgres CRUD
3 HTTP samples	~1h	Two test backends
4 gRPC sample	~2h	Proto + server
5 Runtime loader	~2h	Gateway reads DB every N seconds
6 HTTP router	~3h	Dynamic reverse proxy per route
7 gRPC proxy	~4h	Transparent forwarding
8 Admin handlers	~3h	Real `/admin/v1/routes` + `/upstreams`
9 Docker	~2h	Full compose stack
10 Test	~1h	curl + grpcurl

Recommended order: 1 → 2 → 3 → 6 → 9 → test HTTP → 4 → 7 → test gRPC → 8 → Admin UI.

---

4. Phase 1 — Database Schema

4.1 Create migration files

File: `migrations/0003_upstreams.up.sql`
sql

```
CREATE TABLE upstreams (
    id          UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    name        TEXT NOT NULL UNIQUE,
    target_url  TEXT NOT NULL,
    protocol    TEXT NOT NULL CHECK (protocol IN ('http', 'grpc')),
    health_path TEXT,
    enabled     BOOLEAN NOT NULL DEFAULT TRUE,
    created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at  TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_upstreams_name ON upstreams(name);
CREATE INDEX idx_upstreams_enabled ON upstreams(enabled);
```

File: `migrations/0003_upstreams.down.sql`
sql

```
DROP TABLE IF EXISTS upstreams;
```

File: `migrations/0005_routes_extend.up.sql`

sql

```
-- Extend existing routes table (from 0002_routes.up.sql)
ALTER TABLE routes
    ADD COLUMN IF NOT EXISTS upstream_id UUID REFERENCES upstreams(id) ON DELETE SET NULL,
    ADD COLUMN IF NOT EXISTS protocol TEXT NOT NULL DEFAULT 'http'
        CHECK (protocol IN ('http', 'grpc')),
    ADD COLUMN IF NOT EXISTS match_type TEXT NOT NULL DEFAULT 'prefix'
        CHECK (match_type IN ('exact', 'prefix')),
    ADD COLUMN IF NOT EXISTS enabled BOOLEAN NOT NULL DEFAULT TRUE,
    ADD COLUMN IF NOT EXISTS auth_required BOOLEAN NOT NULL DEFAULT TRUE,
    ADD COLUMN IF NOT EXISTS rate_limit_rpm INT NOT NULL DEFAULT 0,
    ADD COLUMN IF NOT EXISTS updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW();

CREATE INDEX IF NOT EXISTS idx_routes_path ON routes(path);
CREATE INDEX IF NOT EXISTS idx_routes_enabled ON routes(enabled);
```

File: `migrations/0005_routes_extend.down.sql`

sql

```
ALTER TABLE routes
    DROP COLUMN IF EXISTS upstream_id,
    DROP COLUMN IF EXISTS protocol,
    DROP COLUMN IF EXISTS match_type,
    DROP COLUMN IF EXISTS enabled,
    DROP COLUMN IF EXISTS auth_required,
    DROP COLUMN IF EXISTS rate_limit_rpm,
    DROP COLUMN IF EXISTS updated_at;
```

4.2 Seed data for local testing

File: `scripts/seed_routes.sql`

sql

```
INSERT INTO upstreams (name, target_url, protocol, health_path) VALUES
  ('http-user-service',  'http://http-user:9001',  'http', '/health'),
  ('http-order-service', 'http://http-order:9002', 'http', '/health'),
  ('grpc-hello-service', 'grpc-hello:50052',         'grpc', NULL)
ON CONFLICT (name) DO NOTHING;

INSERT INTO routes (path, upstream_url, methods, protocol, match_type, enabled)
SELECT '/api/users',  'http://http-user:9001',  ARRAY['GET','POST','PUT','DELETE'], 'http', 'prefix', TRUE
WHERE NOT EXISTS (SELECT 1 FROM routes WHERE path = '/api/users');

INSERT INTO routes (path, upstream_url, methods, protocol, match_type, enabled)
SELECT '/api/orders', 'http://http-order:9002', ARRAY['GET','POST','PUT','DELETE'], 'http', 'prefix', TRUE
WHERE NOT EXISTS (SELECT 1 FROM routes WHERE path = '/api/orders');

INSERT INTO routes (path, upstream_url, methods, protocol, match_type, enabled)
SELECT '/helloworld.v1.Greeter', 'grpc-hello:50052', ARRAY['*'], 'grpc', 'prefix', TRUE
WHERE NOT EXISTS (SELECT 1 FROM routes WHERE path = '/helloworld.v1.Greeter');
```

Run after stack is up:

bash

```
docker exec -i elitegate_postgres psql -U postgres -d elitegate_db < scripts/seed_routes.sql
```

---

5. Phase 2 — Domain Models & Repositories

5.1 Replace `internal/model/route.go`

File: `internal/model/route.go`

go

```
package model

import "time"

type Route struct {
        ID            string
        Path          string
        UpstreamURL   string
        UpstreamID    *string
        Methods       []string
        Protocol      string // "http" | "grpc"
        MatchType     string // "exact" | "prefix"
        Enabled       bool
        AuthRequired  bool
        RateLimitRPM  int
        CreatedAt     time.Time
        UpdatedAt     time.Time
}

type Upstream struct {
        ID         string
        Name       string
        TargetURL  string
        Protocol   string
        HealthPath string
        Enabled    bool
        CreatedAt  time.Time
        UpdatedAt  time.Time
}
```

5.2 Route repository

File: `internal/storage/route_repo.go` (NEW)

go

```
package storage

import (
        "context"
        "database/sql"
        "errors"
        "time"

        "github.com/lib/pq"

        "elitegate/internal/model"
)

type RouteRepo struct {
        db *sql.DB
}

func NewRouteRepo(db *sql.DB) *RouteRepo {
        return &RouteRepo{db: db}
}

func (r *RouteRepo) ListEnabled(ctx context.Context) ([]model.Route, error) {
        const q = `
                SELECT id, path, upstream_url, upstream_id, methods, protocol,
                       match_type, enabled, auth_required, rate_limit_rpm,
                       created_at, updated_at
                FROM routes
                WHERE enabled = TRUE
                ORDER BY length(path) DESC
        `
        rows, err := r.db.QueryContext(ctx, q)
        if err != nil {
                return nil, err
        }
        defer rows.Close()

        var out []model.Route
        for rows.Next() {
                rt, err := scanRoute(rows)
                if err != nil {
                        return nil, err
                }
                out = append(out, rt)
        }
        return out, rows.Err()
}

func (r *RouteRepo) ListAll(ctx context.Context) ([]model.Route, error) {
        const q = `
                SELECT id, path, upstream_url, upstream_id, methods, protocol,
                       match_type, enabled, auth_required, rate_limit_rpm,
                       created_at, updated_at
                FROM routes
                ORDER BY path ASC
        `
        rows, err := r.db.QueryContext(ctx, q)
        if err != nil {
                return nil, err
        }
        defer rows.Close()

        var out []model.Route
        for rows.Next() {
                rt, err := scanRoute(rows)
                if err != nil {
                        return nil, err
                }
                out = append(out, rt)
        }
        return out, rows.Err()
}
```

```
func (r *RouteRepo) Create(ctx context.Context, rt *model.Route) error {
	const q = `
		INSERT INTO routes (path, upstream_url, methods, protocol, match_type, enabled, auth_required,
rate_limit_rpm)
		VALUES ($1, $2, $3, $4, $5, $6, $7, $8)
		RETURNING id, created_at, updated_at
	`
	return r.db.QueryRowContext(ctx, q,
		rt.Path, rt.UpstreamURL, pq.Array(rt.Methods), rt.Protocol,
		rt.MatchType, rt.Enabled, rt.AuthRequired, rt.RateLimitRPM,
	).Scan(&rt.ID, &rt.CreatedAt, &rt.UpdatedAt)
}

func (r *RouteRepo) Delete(ctx context.Context, id string) error {
	res, err := r.db.ExecContext(ctx, `DELETE FROM routes WHERE id = $1`, id)
	if err != nil {
		return err
	}
	n, _ := res.RowsAffected()
	if n == 0 {
		return sql.ErrNoRows
	}
	return nil
}

type rowScanner interface {
	Scan(dest ...any) error
}

func scanRoute(s rowScanner) (model.Route, error) {
	var rt model.Route
	var upstreamID sql.NullString
	err := s.Scan(
		&rt.ID, &rt.Path, &rt.UpstreamURL, &upstreamID, pq.Array(&rt.Methods),
		&rt.Protocol, &rt.MatchType, &rt.Enabled, &rt.AuthRequired, &rt.RateLimitRPM,
		&rt.CreatedAt, &rt.UpdatedAt,
	)
	if upstreamID.Valid {
		rt.UpstreamID = &upstreamID.String
	}
	return rt, err
}

var ErrRouteNotFound = errors.New("route not found")
```

5.3 Upstream repository

File: `internal/storage/upstream_repo.go` (NEW)

go

```
package storage

import (
        "context"
        "database/sql"

        "elitegate/internal/model"
)

type UpstreamRepo struct {
        db *sql.DB
}

func NewUpstreamRepo(db *sql.DB) *UpstreamRepo {
        return &UpstreamRepo{db: db}
}

func (r *UpstreamRepo) ListAll(ctx context.Context) ([]model.Upstream, error) {
        const q = `
                SELECT id, name, target_url, protocol, COALESCE(health_path,''),
                       enabled, created_at, updated_at
                FROM upstreams
                ORDER BY name ASC
        `
        rows, err := r.db.QueryContext(ctx, q)
        if err != nil {
                return nil, err
        }
        defer rows.Close()

        var out []model.Upstream
        for rows.Next() {
                var u model.Upstream
                if err := rows.Scan(
                        &u.ID, &u.Name, &u.TargetURL, &u.Protocol, &u.HealthPath,
                        &u.Enabled, &u.CreatedAt, &u.UpdatedAt,
                ); err != nil {
                        return nil, err
                }
                out = append(out, u)
        }
        return out, rows.Err()
}

func (r *UpstreamRepo) Create(ctx context.Context, u *model.Upstream) error {
        const q = `
                INSERT INTO upstreams (name, target_url, protocol, health_path, enabled)
                VALUES ($1, $2, $3, $4, $5)
                RETURNING id, created_at, updated_at
        `
        return r.db.QueryRowContext(ctx, q,
                u.Name, u.TargetURL, u.Protocol, u.HealthPath, u.Enabled,
        ).Scan(&u.ID, &u.CreatedAt, &u.UpdatedAt)
}
```

---

6. Phase 3 — Sample HTTP Services

Move test backends off port `9090` (admin conflict).

6.1 User service

File: `cmd/samples/http-user/main.go` (NEW)

go

```
package main

import (
        "encoding/json"
        "net/http"
        "os"

        "github.com/rs/zerolog"
)

func main() {
        logger := zerolog.New(os.Stdout).With().Timestamp().Str("service", "http-user").Logger()
        mux := http.NewServeMux()

        mux.HandleFunc("/health", func(w http.ResponseWriter, r *http.Request) {
                w.WriteHeader(http.StatusOK)
                _, _ = w.Write([]byte(`{"status":"ok"}`))
        })

        mux.HandleFunc("/", func(w http.ResponseWriter, r *http.Request) {
                w.Header().Set("Content-Type", "application/json")
                _ = json.NewEncoder(w).Encode(map[string]any{
                        "service":      "http-user-service",
                        "method":       r.Method,
                        "path":         r.URL.Path,
                        "forwarded_by": r.Header.Get("X-Gateway"),
                })
        })

        addr := ":9001"
        logger.Info().Str("addr", addr).Msg("http-user-service listening")
        if err := http.ListenAndServe(addr, mux); err != nil {
                logger.Fatal().Err(err).Msg("server failed")
        }
}
```

6.2 Order service

File: `cmd/samples/http-order/main.go` (NEW)

go

```
package main

import (
        "encoding/json"
        "net/http"
        "os"

        "github.com/rs/zerolog"
)

func main() {
        logger := zerolog.New(os.Stdout).With().Timestamp().Str("service", "http-order").Logger()
        mux := http.NewServeMux()

        mux.HandleFunc("/health", func(w http.ResponseWriter, r *http.Request) {
                w.WriteHeader(http.StatusOK)
                _, _ = w.Write([]byte(`{"status":"ok"}`))
        })

        mux.HandleFunc("/", func(w http.ResponseWriter, r *http.Request) {
                w.Header().Set("Content-Type", "application/json")
                _ = json.NewEncoder(w).Encode(map[string]any{
                        "service":      "http-order-service",
                        "method":       r.Method,
                        "path":         r.URL.Path,
                        "forwarded_by": r.Header.Get("X-Gateway"),
                })
        })

        addr := ":9002"
        logger.Info().Str("addr", addr).Msg("http-order-service listening")
        if err := http.ListenAndServe(addr, mux); err != nil {
                logger.Fatal().Err(err).Msg("server failed")
        }
}
```

---

7. Phase 4 — Sample gRPC Service

7.1 Proto definition

File: `api/proto/helloworld/v1/hello.proto` (NEW)

protobuf

```
syntax = "proto3";

package helloworld.v1;

option go_package = "elitegate/gen/helloworld/v1;helloworldv1";

service Greeter {
  rpc SayHello (HelloRequest) returns (HelloReply);
}

message HelloRequest {
  string name = 1;
}

message HelloReply {
  string message = 1;
}
```

Generate Go code (run from repo root):

bash

```
go install google.golang.org/protobuf/cmd/protoc-gen-go@latest
go install google.golang.org/grpc/cmd/protoc-gen-go-grpc@latest

protoc \
  --go_out=. --go_opt=paths=source_relative \
  --go-grpc_out=. --go-grpc_opt=paths=source_relative \
  api/proto/helloworld/v1/hello.proto
```

This creates:

- `api/proto/helloworld/v1/hello.pb.go`

- `api/proto/helloworld/v1/hello_grpc.pb.go`

Add to `go.mod`:

bash

```
go get google.golang.org/grpc@latest
go mod tidy
```

7.2 gRPC server

File: `cmd/samples/grpc-hello/main.go` (NEW)
go

```
package main

import (
        "context"
        "fmt"
        "net"
        "os"

        pb "elitegate/api/proto/helloworld/v1"

        "github.com/rs/zerolog"
        "google.golang.org/grpc"
)

type greeterServer struct {
        pb.UnimplementedGreeterServer
}

func (s *greeterServer) SayHello(ctx context.Context, req *pb.HelloRequest) (*pb.HelloReply, error) {
        name := req.GetName()
        if name == "" {
                name = "world"
        }
        return &pb.HelloReply{Message: fmt.Sprintf("Hello %s from grpc-hello-service", name)}, nil
}

func main() {
        logger := zerolog.New(os.Stdout).With().Timestamp().Str("service", "grpc-hello").Logger()
        lis, err := net.Listen("tcp", ":50052")
        if err != nil {
                logger.Fatal().Err(err).Msg("listen failed")
        }

        s := grpc.NewServer()
        pb.RegisterGreeterServer(s, &greeterServer{})

        logger.Info().Str("addr", ":50052").Msg("grpc-hello-service listening")
        if err := s.Serve(lis); err != nil {
                logger.Fatal().Err(err).Msg("serve failed")
        }
}
```

---

8. Phase 5 — Gateway Runtime (Route Loader)

8.1 Runtime snapshot

File: `internal/gateway/runtime/config.go` (REPLACE stub)

go

```
package runtime

import "elitegate/internal/model"

type Snapshot struct {
        Routes []model.Route
}

func (s Snapshot) RouteCount() int {
        return len(s.Routes)
}
```

8.2 Loader with periodic reload

File: `internal/gateway/runtime/loader.go` (REPLACE stub)

go

```
package runtime

import (
        "context"
        "sync"
        "time"

        "github.com/rs/zerolog"

        "elitegate/internal/storage"
)

type Loader struct {
        repo     *storage.RouteRepo
        logger   zerolog.Logger
        interval time.Duration

        mu       sync.RWMutex
        snapshot Snapshot
}

func NewLoader(repo *storage.RouteRepo, logger zerolog.Logger, interval time.Duration) *Loader {
        return &Loader{
                repo:     repo,
                logger:   logger,
                interval: interval,
        }
}

func (l *Loader) Start(ctx context.Context) error {
        if err := l.reload(ctx); err != nil {
                return err
        }
        go l.loop(ctx)
        return nil
}

func (l *Loader) loop(ctx context.Context) {
        ticker := time.NewTicker(l.interval)
        defer ticker.Stop()
        for {
                select {
                case <-ctx.Done():
                        return
                case <-ticker.C:
                        if err := l.reload(ctx); err != nil {
                                l.logger.Error().Err(err).Msg("route reload failed")
                        }
                }
        }
}

func (l *Loader) reload(ctx context.Context) error {
        routes, err := l.repo.ListEnabled(ctx)
        if err != nil {
                return err
        }
        l.mu.Lock()
        l.snapshot = Snapshot{Routes: routes}
        l.mu.Unlock()
        l.logger.Info().Int("routes", len(routes)).Msg("gateway routes reloaded")
        return nil
}

func (l *Loader) Current() Snapshot {
        l.mu.RLock()
        defer l.mu.RUnlock()
        return l.snapshot
}
```

8.3 Wire loader in gateway app

File: `internal/gateway/app/app.go` — add fields and start loader:

go

```
// Add imports: context, time, runtime package, storage

type App struct {
        // ... existing fields ...
        RouteLoader *runtime.Loader
}

func StartApp(cfg *config.Config) (*App, error) {
        // ... after db connect ...
        routeRepo := storage.NewRouteRepo(db)
        loader := runtime.NewLoader(routeRepo, logger, 10*time.Second)
        if err := loader.Start(context.Background()); err != nil {
                return nil, fmt.Errorf("route loader: %w", err)
        }

        router, err := gatewayRouter.NewRouter(logger, rdb, cfg, loader)
        // ... rest unchanged, attach loader to App struct ...
}
```

---

9. Phase 6 — Dynamic HTTP Router

9.1 Route matcher

File: `internal/gateway/router/matcher.go` (REPLACE stub)

go

```
package router

import (
        "strings"

        "elitegate/internal/model"
)

func MatchHTTP(path string, routes []model.Route) *model.Route {
        var best *model.Route
        bestLen := -1

        for i := range routes {
                rt := &routes[i]
                if rt.Protocol != "http" || !rt.Enabled {
                        continue
                }
                switch rt.MatchType {
                case "exact":
                        if path == rt.Path && len(rt.Path) > bestLen {
                                best, bestLen = rt, len(rt.Path)
                        }
                default: // prefix
                        if strings.HasPrefix(path, rt.Path) && len(rt.Path) > bestLen {
                                best, bestLen = rt, len(rt.Path)
                        }
                }
        }
        return best
}
```

9.2 Dynamic proxy handler

File: `internal/gateway/handler/proxy_handler.go` (REPLACE stub)

go

```
package handler

import (
        "net/http"
        "sync"

        "elitegate/internal/gateway/proxy"
        "elitegate/internal/gateway/router"
        "elitegate/internal/gateway/runtime"
)

type DynamicProxy struct {
        loader *runtime.Loader

        mu      sync.Mutex
        proxies map[string]*proxy.ReverseProxy
}

func NewDynamicProxy(loader *runtime.Loader) *DynamicProxy {
        return &DynamicProxy{
                loader:  loader,
                proxies: make(map[string]*proxy.ReverseProxy),
        }
}

func (d *DynamicProxy) ServeHTTP(w http.ResponseWriter, r *http.Request) {
        snap := d.loader.Current()
        rt := router.MatchHTTP(r.URL.Path, snap.Routes)
        if rt == nil {
                http.Error(w, `{"error":"route not found"}`, http.StatusNotFound)
                return
        }

        p, err := d.getProxy(rt.UpstreamURL)
        if err != nil {
                http.Error(w, `{"error":"bad upstream"}`, http.StatusBadGateway)
                return
        }
        p.ServeHTTP(w, r)
}

func (d *DynamicProxy) getProxy(target string) (*proxy.ReverseProxy, error) {
        d.mu.Lock()
        defer d.mu.Unlock()
        if p, ok := d.proxies[target]; ok {
                return p, nil
        }
        p, err := proxy.New(target)
        if err != nil {
                return nil, err
        }
        d.proxies[target] = p
        return p, nil
}
```

9.3 Replace gateway router (CRITICAL)

File: `internal/gateway/router.go` — replace admin proxy with dynamic handler:

go

```
package gateway

import (
        "net/http"

        "elitegate/internal/config"
        "elitegate/internal/gateway/handler"
        "elitegate/internal/gateway/middleware"
        "elitegate/internal/gateway/runtime"
        "elitegate/internal/ratelimit"

        "github.com/redis/go-redis/v9"
        "github.com/rs/zerolog"
)

func NewRouter(
        logger zerolog.Logger,
        rdb *redis.Client,
        cfg *config.Config,
        loader *runtime.Loader,
) (http.Handler, error) {
        dynamic := handler.NewDynamicProxy(loader)

        rpm := cfg.RateLimit.RequestsPerMinute
        memFallback := ratelimit.NewMemoryLimiter(rpm)
        limiter := ratelimit.NewRedisLimiter(rdb, rpm, memFallback)
        rlMiddleware := middleware.NewRateLimitMiddleware(limiter)

        mux := http.NewServeMux()

        mux.HandleFunc("/health", func(w http.ResponseWriter, r *http.Request) {
                w.Header().Set("Content-Type", "application/json")
                _, _ = w.Write([]byte(`{"status":"ok","service":"gateway"}`))
        })
        mux.HandleFunc("/ready", func(w http.ResponseWriter, r *http.Request) {
                w.Header().Set("Content-Type", "application/json")
                _, _ = w.Write([]byte(`{"status":"ready"}`))
        })

        apiHandler := middleware.Chain(
                dynamic,
                middleware.IPFilter,
                middleware.Auth(cfg.Auth.JWTSecret),
                rlMiddleware.Middleware,
        )
        mux.Handle("/api/", apiHandler)

        return middleware.Chain(
                mux,
                middleware.Recovery(logger),
                middleware.RequestLogger(logger),
        ), nil
}
```

Delete or stop using the hardcoded routes in `internal/gateway/router/router.go` once DB routing works.

---

10. Phase 7 — Transparent gRPC Proxy

Add dependency:

bash

```
go get google.golang.org/grpc@latest
```

10.1 gRPC proxy implementation

File: `internal/gateway/proxy/grpc_proxy.go` (REPLACE empty file)

go

```
package proxy

import (
        "context"
        "io"
        "strings"

        "google.golang.org/grpc"
        "google.golang.org/grpc/codes"
        "google.golang.org/grpc/metadata"
        "google.golang.org/grpc/status"
)

// TransparentHandler forwards unknown RPCs to backendAddr using gRPC transparent proxying.
func TransparentHandler(backendAddr string) grpc.StreamHandler {
        return func(srv interface{}, stream grpc.ServerStream) error {
                fullMethod, ok := grpc.MethodFromServerStream(stream)
                if !ok {
                        return status.Error(codes.Internal, "method not found")
                }

                ctx := stream.Context()
                md, _ := metadata.FromIncomingContext(ctx)
                outCtx := metadata.NewOutgoingContext(ctx, md)

                conn, err := grpc.DialContext(ctx, backendAddr,
                        grpc.WithCodec(proxyCodec{}),
                        grpc.WithInsecure(), // use TLS in production
                )
                if err != nil {
                        return status.Errorf(codes.Unavailable, "dial backend: %v", err)
                }
                defer conn.Close()

                clientStream, err := grpc.NewClientStream(outCtx, &grpc.StreamDesc{
                        StreamName:    fullMethod,
                        ServerStreams: true,
                        ClientStreams: true,
                }, conn, fullMethod)
                if err != nil {
                        return status.Errorf(codes.Internal, "client stream: %v", err)
                }

                errCh := make(chan error, 2)
                go func() { errCh <- copyStream(clientStream, stream) }()
                go func() { errCh <- copyStream(stream, clientStream) }()

                for i := 0; i < 2; i++ {
                        if err := <-errCh; err != nil && err != io.EOF {
                                return err
                        }
                }
                return nil
        }
}

func copyStream(dst grpc.Stream, src grpc.Stream) error {
        for {
                msg := &rawFrame{}
                if err := src.RecvMsg(msg); err != nil {
                        return err
                }
                if err := dst.SendMsg(msg); err != nil {
                        return err
                }
        }
}

type rawFrame struct{ data []byte }

func (r *rawFrame) MarshalVT() ([]byte, error)   { return r.data, nil }
```

```
func (r *rawFrame) UnmarshalVT(b []byte) error    { r.data = append([]byte(nil), b...); return nil }
func (r *rawFrame) SizeVT() int                  { return len(r.data) }

type proxyCodec struct{}

func (proxyCodec) Marshal(v interface{}) ([]byte, error) {
        if f, ok := v.(*rawFrame); ok {
                return f.data, nil
        }
        return nil, status.Error(codes.Internal, "invalid frame")
}
func (proxyCodec) Unmarshal(data []byte, v interface{}) error {
        if f, ok := v.(*rawFrame); ok {
                f.data = append([]byte(nil), data...)
                return nil
        }
        return status.Error(codes.Internal, "invalid frame")
}
func (proxyCodec) Name() string { return "proxy" }

// ResolveGRPCBackend picks upstream host from route table (longest prefix on service name).
func ResolveGRPCBackend(fullMethod string, upstreamByPrefix map[string]string) (string, bool) {
        // fullMethod like "/helloworld.v1.Greeter/SayHello"
        service := strings.TrimPrefix(fullMethod, "/")
        if i := strings.Index(service, "/"); i > 0 {
                service = service[:i]
        }
        var best string
        bestLen := -1
        for prefix, addr := range upstreamByPrefix {
                if strings.HasPrefix(service, prefix) && len(prefix) > bestLen {
                        best, bestLen = addr, len(prefix)
                }
        }
        return best, best != ""
}
```

> **Production note:** For a portfolio MVP, consider `github.com/mwitkow/grpc-proxy` or Envoy sidecar later. The above shows the transparent-proxy pattern from your design doc.

10.2 gRPC gateway server

File: `internal/gateway/server/grpc_server.go` (NEW)
go

```
package server

import (
	"net"

	"github.com/rs/zerolog"
	"google.golang.org/grpc"
	"google.golang.org/grpc/codes"
	"google.golang.org/grpc/status"

	"elitegate/internal/gateway/proxy"
	"elitegate/internal/gateway/runtime"
)

type GRPCGateway struct {
	logger zerolog.Logger
	loader *runtime.Loader
	port   string
}

func NewGRPCGateway(logger zerolog.Logger, loader *runtime.Loader, port string) *GRPCGateway {
	return &GRPCGateway{logger: logger, loader: loader, port: port}
}

func (g *GRPCGateway) Start() error {
	lis, err := net.Listen("tcp", g.port)
	if err != nil {
		return err
	}
	s := grpc.NewServer(
		grpc.UnknownServiceHandler(func(srv interface{}, stream grpc.ServerStream) error {
			fullMethod, _ := grpc.MethodFromServerStream(stream)
			backends := g.buildGRPCBackends()
			addr, ok := proxy.ResolveGRPCBackend(fullMethod, backends)
			if !ok {
				return status.Error(codes.NotFound, "no grpc route")
			}
			return proxy.TransparentHandler(addr)(srv, stream)
		}),
	)
	g.logger.Info().Str("port", g.port).Msg("gRPC gateway listening")
	return s.Serve(lis)
}

func (g *GRPCGateway) buildGRPCBackends() map[string]string {
	out := make(map[string]string)
	for _, rt := range g.loader.Current().Routes {
		if rt.Protocol == "grpc" && rt.Enabled {
			out[rt.Path] = rt.UpstreamURL
		}
	}
	return out
}
```

Start gRPC server in `internal/gateway/app/app.go` in a goroutine on `:50051`.

---

11. Phase 8 — Admin API (Control Plane)

Replace stub handlers in `internal/admin/router.go` with real handlers.

11.1 Route handler

File: `internal/admin/handler/route_handler.go` (NEW)

go

```
package handler

import (
	"net/http"

	"github.com/gin-gonic/gin"

	"elitegate/internal/model"
	"elitegate/internal/storage"
)

type RouteHandler struct {
	repo *storage.RouteRepo
}

func NewRouteHandler(repo *storage.RouteRepo) *RouteHandler {
	return &RouteHandler{repo: repo}
}

func (h *RouteHandler) List(c *gin.Context) {
	routes, err := h.repo.ListAll(c.Request.Context())
	if err != nil {
		c.JSON(http.StatusInternalServerError, gin.H{"error": err.Error()})
		return
	}
	c.JSON(http.StatusOK, gin.H{"routes": routes})
}

type createRouteRequest struct {
	Path         string   `json:"path" binding:"required"`
	UpstreamURL  string   `json:"upstream_url" binding:"required"`
	Methods      []string `json:"methods" binding:"required"`
	Protocol     string   `json:"protocol"`
	MatchType    string   `json:"match_type"`
	Enabled      bool     `json:"enabled"`
	AuthRequired bool     `json:"auth_required"`
	RateLimitRPM int      `json:"rate_limit_rpm"`
}

func (h *RouteHandler) Create(c *gin.Context) {
	var req createRouteRequest
	if err := c.ShouldBindJSON(&req); err != nil {
		c.JSON(http.StatusBadRequest, gin.H{"error": err.Error()})
		return
	}
	if req.Protocol == "" {
		req.Protocol = "http"
	}
	if req.MatchType == "" {
		req.MatchType = "prefix"
	}
	rt := &model.Route{
		Path: req.Path, UpstreamURL: req.UpstreamURL, Methods: req.Methods,
		Protocol: req.Protocol, MatchType: req.MatchType, Enabled: req.Enabled,
		AuthRequired: req.AuthRequired, RateLimitRPM: req.RateLimitRPM,
	}
	if err := h.repo.Create(c.Request.Context(), rt); err != nil {
		c.JSON(http.StatusInternalServerError, gin.H{"error": err.Error()})
		return
	}
	c.JSON(http.StatusCreated, gin.H{"route": rt})
}

func (h *RouteHandler) Delete(c *gin.Context) {
	if err := h.repo.Delete(c.Request.Context(), c.Param("id")); err != nil {
		c.JSON(http.StatusNotFound, gin.H{"error": "not found"})
		return
	}
	c.JSON(http.StatusOK, gin.H{"message": "deleted"})
}
```

11.2 Wire in admin router

File: `internal/admin/router.go` — replace stub `listRoutesHandler` etc.:
go

```
routeRepo := storage.NewRouteRepo(db)
routeHandler := handler.NewRouteHandler(routeRepo)

routes := v1.Group("/routes")
{
    routes.GET("", routeHandler.List)
    routes.POST("", routeHandler.Create)
    routes.DELETE("/:id", routeHandler.Delete)
}

upstreamRepo := storage.NewUpstreamRepo(db)
upstreamHandler := handler.NewUpstreamHandler(upstreamRepo)

upstreams := v1.Group("/upstreams")
{
    upstreams.GET("", upstreamHandler.List)
    upstreams.POST("", upstreamHandler.Create)
}
```

Gateway reloads routes every 10s — **no gateway restart** needed after admin changes.

---

12. Phase 9 — Docker Compose (Full Stack)

File: `deploy/docker-compose.yml` — add sample services (append under `services:`):
yaml

```
  http-user:
    build:
      context: ..
      dockerfile: deploy/docker/sample-http.Dockerfile
      args:
        SAMPLE_CMD: ./cmd/samples/http-user
    container_name: elitegate_http_user
    networks:
      - elitegate_net

  http-order:
    build:
      context: ..
      dockerfile: deploy/docker/sample-http.Dockerfile
      args:
        SAMPLE_CMD: ./cmd/samples/http-order
    container_name: elitegate_http_order
    networks:
      - elitegate_net

  grpc-hello:
    build:
      context: ..
      dockerfile: deploy/docker/sample-grpc.Dockerfile
    container_name: elitegate_grpc_hello
    networks:
      - elitegate_net
```

File: `deploy/docker/sample-http.Dockerfile` (NEW)
dockerfile

```
FROM golang:1.25-alpine AS builder
WORKDIR /app
ARG SAMPLE_CMD
COPY go.mod go.sum ./
RUN go mod download
COPY . .
RUN CGO_ENABLED=0 go build -o /sample ${SAMPLE_CMD}

FROM alpine:3.20
COPY --from=builder /sample /sample
CMD ["/sample"]
```

File: `deploy/docker/sample-grpc.Dockerfile` (NEW)
dockerfile

```
FROM golang:1.25-alpine AS builder
WORKDIR /app
COPY go.mod go.sum ./
RUN go mod download
COPY . .
RUN CGO_ENABLED=0 go build -o /sample ./cmd/samples/grpc-hello

FROM alpine:3.20
COPY --from=builder /sample /sample
EXPOSE 50052
CMD ["/sample"]
```

Update `gateway` service `depends_on`:

yaml

```
    depends_on:
      postgres:
        condition: service_healthy
      redis:
        condition: service_healthy
      http-user:
        condition: service_started
      http-order:
        condition: service_started
      grpc-hello:
        condition: service_started
```

Add to gateway env (`.env`):

env

```
GRPC_GATEWAY_PORT=:50051
ROUTE_RELOAD_INTERVAL=10s
```

Expose gRPC port on gateway container:

yaml

```
    ports:
      - "8080:8080"
      - "50051:50051"
```

---

13. Phase 10 — Makefile & Config

File: `Makefile` — add targets:
makefile

```
HTTP_USER_MAIN  := ./cmd/samples/http-user
HTTP_ORDER_MAIN := ./cmd/samples/http-order
GRPC_HELLO_MAIN := ./cmd/samples/grpc-hello

run-http-user:
        go run $(HTTP_USER_MAIN)

run-http-order:
        go run $(HTTP_ORDER_MAIN)

run-grpc-hello:
        go run $(GRPC_HELLO_MAIN)

seed-routes:
        docker exec -i elitegate_postgres psql -U postgres -d elitegate_db < scripts/seed_routes.sql
```

File: `internal/config/config.go` — add:
go

```
GRPCGatewayPort     string `mapstructure:"grpc_gateway_port"`
RouteReloadInterval string `mapstructure:"route_reload_interval"`
```

File: `internal/config/config.yaml` — add:
yaml

```
server:
  grpc_gateway_port: ":50051"
  route_reload_interval: "10s"
```

---

14. Testing Checklist

14.1 Start stack

bash

```
make infra-up
make docker-up
make seed-routes
```

14.2 Generate JWT

bash

```
make token CLIENT=demo-client ROLE=client
```

14.3 HTTP through gateway

bash

```
# User service
curl -s -H "Authorization: Bearer <TOKEN>" \
  http://localhost:8080/api/users/42 | jq

# Order service
curl -s -H "Authorization: Bearer <TOKEN>" \
  http://localhost:8080/api/orders/99 | jq

# Expect forwarded_by: elitegate/1.0
```

14.4 gRPC through gateway

bash

```
grpcurl -plaintext \
  -d '{"name":"EliteGuard"}' \
  localhost:50051 helloworld.v1.Greeter/SayHello
```

14.5 Admin CRUD (control plane)

bash

```
# Login
curl -s -X POST http://localhost:9090/admin/login \
  -H "Content-Type: application/json" \
  -d '{"username":"admin","password":"your-password"}'

# List routes (use access_token from login)
curl -s http://localhost:9090/admin/v1/routes \
  -H "Authorization: Bearer <ADMIN_TOKEN>"
```

14.6 Verify DB

bash

```
docker exec -it elitegate_postgres psql -U postgres -d elitegate_db \
  -c "SELECT path, upstream_url, protocol, enabled FROM routes;"
```

14.7 Expected failures (before implementation)

Test	If not implemented yet
`/api/users` via gateway	404 or proxies to admin
gRPC via `:50051`	connection refused
Admin POST route	returns `"message":"created"` stub only

---

16. File Checklist

Use this as your implementation tracker.

Status	File	Action
☐	`migrations/0003_upstreams.up.sql`	CREATE
☐	`migrations/0003_upstreams.down.sql`	CREATE
☐	`migrations/0005_routes_extend.up.sql`	CREATE
☐	`migrations/0005_routes_extend.down.sql`	CREATE
☐	`scripts/seed_routes.sql`	CREATE
☐	`internal/model/route.go`	REPLACE
☐	`internal/model/upstream.go`	CREATE
☐	`internal/storage/route_repo.go`	CREATE
☐	`internal/storage/upstream_repo.go`	CREATE
☐	`cmd/samples/http-user/main.go`	CREATE
☐	`cmd/samples/http-order/main.go`	CREATE
☐	`api/proto/helloworld/v1/hello.proto`	CREATE
☐	`cmd/samples/grpc-hello/main.go`	CREATE
☐	`internal/gateway/runtime/config.go`	REPLACE
☐	`internal/gateway/runtime/loader.go`	REPLACE
☐	`internal/gateway/router/matcher.go`	REPLACE
☐	`internal/gateway/handler/proxy_handler.go`	REPLACE
☐	`internal/gateway/router.go`	REPLACE
☐	`internal/gateway/proxy/grpc_proxy.go`	REPLACE
☐	`internal/gateway/server/grpc_server.go`	CREATE
☐	`internal/gateway/app/app.go`	MODIFY
☐	`internal/admin/handler/route_handler.go`	CREATE
☐	`internal/admin/handler/upstream_handler.go`	CREATE
☐	`internal/admin/router.go`	MODIFY
☐	`deploy/docker-compose.yml`	MODIFY
☐	`deploy/docker/sample-http.Dockerfile`	CREATE
☐	`deploy/docker/sample-grpc.Dockerfile`	CREATE
☐	`Makefile`	MODIFY
☐	`internal/config/config.yaml`	MODIFY
☐	`go.mod`	ADD grpc deps

Quick Reference — Ports

Service	Port	Purpose
Gateway HTTP	8080	Public API entry
Gateway gRPC	50051	Transparent gRPC proxy
Admin API	9090	Control plane (NOT proxied)
http-user	9001	Sample HTTP backend
http-order	9002	Sample HTTP backend
grpc-hello	50052	Sample gRPC backend
Postgres	5433 (host)	Config storage
Redis	6379	Rate limiting

Summary

1. **HTTP forwarding today** proxies to admin — wrong for production; switch to **DB-backed dynamic router**.
2. **gRPC** needs new deps, sample server, and **transparent proxy** on `:50051`.
3. **DB** works for admin auth; **routes/upstreams** need repositories + real admin handlers.
4. **Admin UI** configures routes; gateway reloads without restart.
5. Follow phases **1** → **6** → **9** first for a working HTTP demo, then add gRPC.

EliteGuard / CoreGuard Gateway — Implementation Guide v1.0
